

TRIAGEM MÉDICA AUTOMÁTICA

Classificando urgência de laudos com pseudo-rotulagem BioBERT + modelo leve — End-to-End

Scikit-Learn

BioBERT

FastAPI

DVC

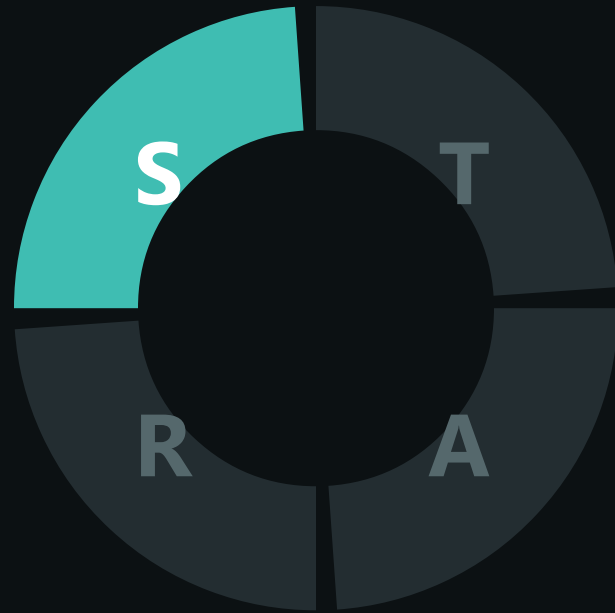
Airflow

Grafana

ONNX

M E D I C A L T R I A G E M L O P S

SITUAÇÃO · STAR : S



O Problema

Contexto

- Hospital precisa triar **laudos médicos em texto** por urgência — **urgente** · **atenção** · **normal** — no momento da chegada
- Requisitos da fase: API leve em Docker, **CI/CD**, orquestração com **Airflow**, **monitoramento** e otimização de latência
- O desafio central é de dados: o dataset público rotula **doenças**, **não urgência**

14.438

abstracts — Medical Abstracts TC Corpus

11.227

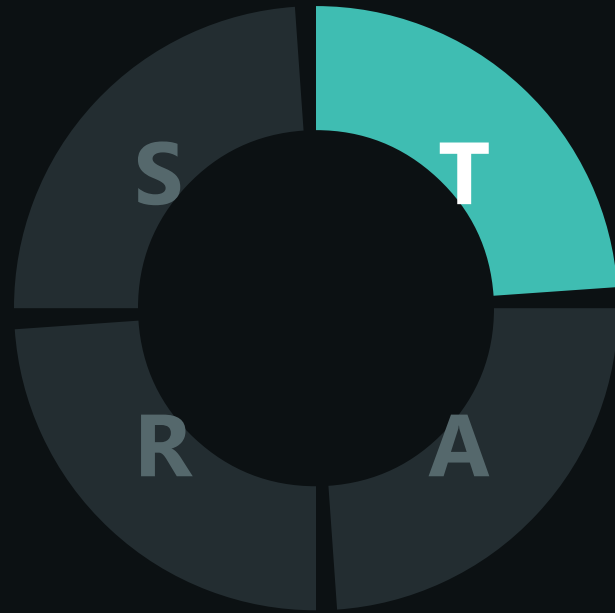
textos únicos após consolidação

5 → 3

categorias de doença no dataset → níveis de urgência exigidos

OBJETIVO: classificar a urgência de um laudo em < 1 ms — com recall alto na classe crítica

TAREFA · STAR : T



Nossa Solução

- 1 BioBERT pré-treinado (urgent / non-urgent) gera **pseudo-rótulos**
- 2 Regra de threshold **0,70** cria a zona de incerteza **atenção**
- 3 Split estratificado **70 / 15 / 15** — DVC, seed 42
- 4 **TF-IDF + Logistic Regression balanced** destila o conhecimento — 0,3 MB
- 5 FastAPI em Docker → **Cloud Run** · Prometheus + Grafana
- 6 Retreino: **DAG Airflow sobre os stages DVC** + quality gate

Metas

Critério de modelo

Recall da classe urgente

Reprodutibilidade

dvc repro exato

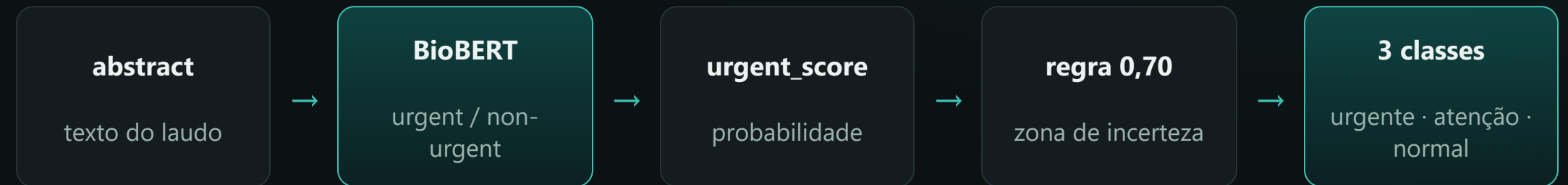
Otimização

Latência mensurável

AÇÃO · STAR : A



Pseudo-rotulagem — a Decisão de Dados

**11.227**

rotulados em ~1 min (GPU)

60% → 3%

truncamento (max_length 256 → 512)

5.000 · 4.873 ·**1.354**

normal · atenção · urgente

⚠ **Auditoria de dados:** a execução parcial original tinha **viés de seleção alfabético** — corrigido com shuffle + corpus completo. E **atenção** é **regra operacional** (incerteza do classificador), não classe clínica — decisão documentada.

AÇÃO · STAR : A



Do Baseline ao Modelo Final

Recall da classe urgente — conjunto de teste

MLP v1 · 720 amostras enviesadas		0,000
MLP v2 · corpus completo		0,591
LogReg balanced — final		0,813

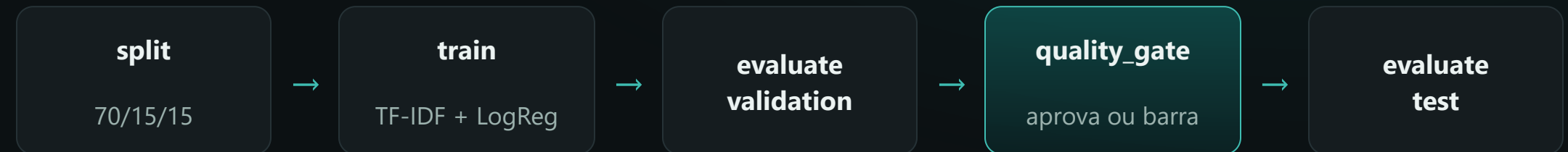
🏆 Por que o LogReg venceu

- Macro F1 **empatado** com o MLP (0,75) — mas `class_weight='balanced'` resolve a classe crítica, que o MLPClassifier não suporta
- 25x menor** (0,3 MB) e mais rápido — ideal para serving leve e exportação ONNX

Numa triagem, o erro caro é deixar passar um caso grave — recall de urgente foi o critério de decisão

AÇÃO · STAR : A

Pipeline DVC + Quality Gate



O gate barra o deploy se...

- Macro F1 < **0,70** na validação
- Recall de **urgente** < **0,70**
- Alguma das 3 classes ausente das predições
- Avaliado **só na validação** — o teste permanece intocado

Versionamento com DVC

- Dados e modelos em bucket **GCS** — o git só carrega código
- **dvc pull** materializa artefatos; **dvc repro** reproduz o pipeline
- Determinístico: seed 42, **uv.lock**, **37 testes**

Um modelo reprovado no gate nunca chega à avaliação final — nem à produção

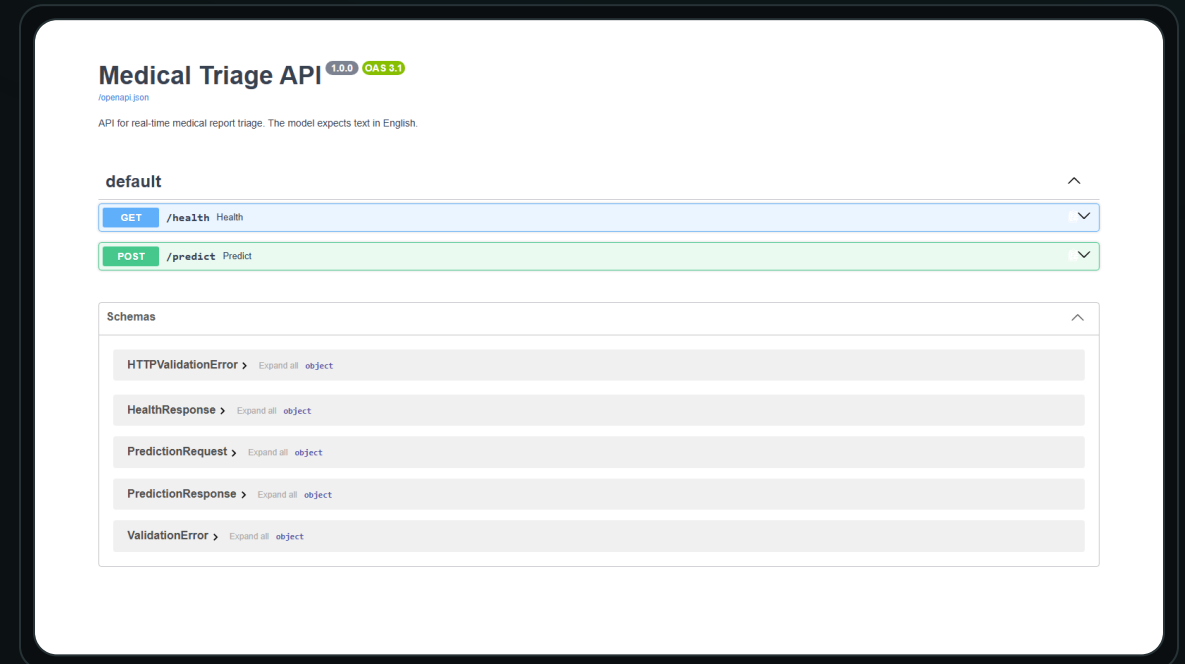
AÇÃO · STAR : A



Produção — API FastAPI no Cloud Run

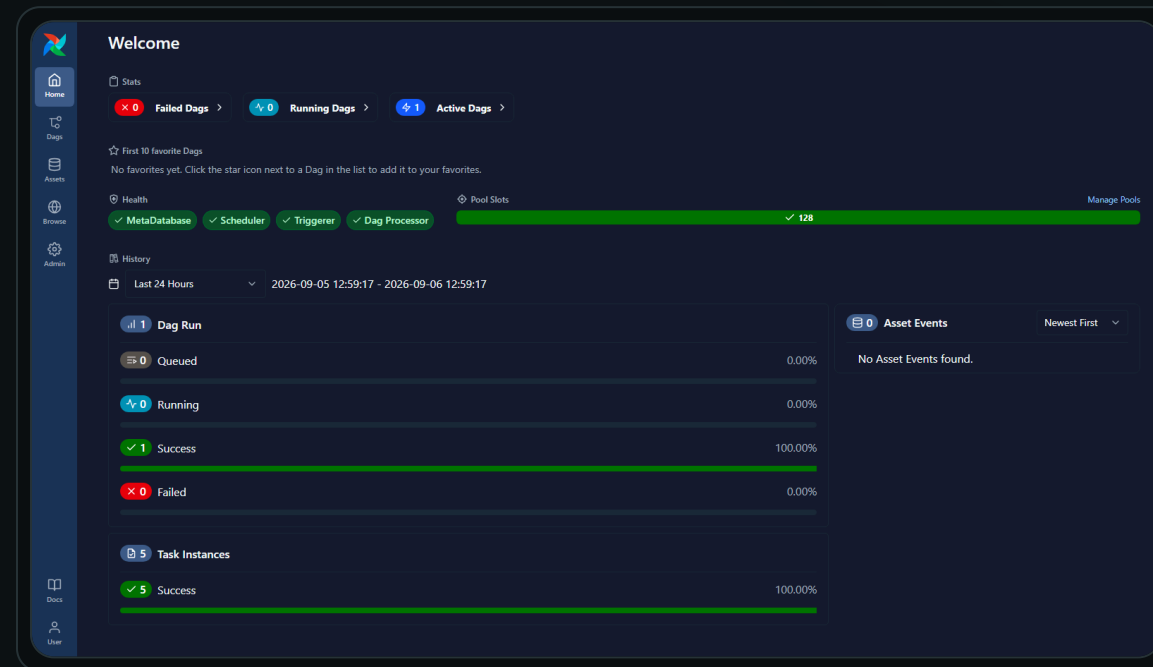
Serving

- `POST /predict` → `{trriage_level, probabilities, model_version}`; `GET /health` como probe
- Modelo carregado **uma vez no startup** — resposta em ~1 ms
- **Sem torch / GPU no serving** — imagem `python:3.11-slim` com extras de dependências separados (api × labeling × airflow)
- Deploy real: **Cloud Run** + Artifact Registry, URL pública



AÇÃO · STAR : A

Orquestração — DAG Airflow

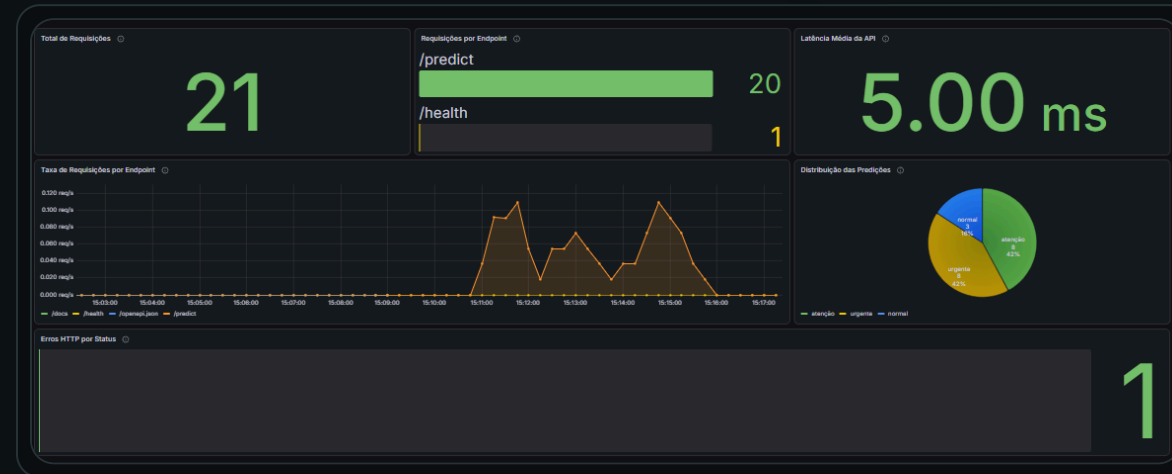


Retreino como código

- DAG `medical_triage_training` = wrapper dos **stages DVC** — fonte única da lógica no `dvc.yaml`
- 5 tasks: split → train → evaluate → **gate** → test
- Execução completa em **28 segundos** — 100% verde
- Batch **isolado** do caminho de inferência: retreino nunca afeta a latência da API

AÇÃO · STAR : A

Observabilidade — Prometheus + Grafana



6 painéis provisionados

- Requisições, taxa por endpoint, latência média, erros HTTP
- Métricas `medical_triage_*` via prometheus-client
- Compose local + scrape da API no Cloud Run

Guard-rail de negócio

O painel de **predições por classe** vigia a série **urgente**: se ela zerar, é regressão do **modelo** — não do serviço. Foi exatamente o defeito do baseline v1.

AÇÃO · STAR : A



Otimização de Latência — ONNX Runtime

Pipeline completo num único grafo — TF-IDF embutido

	SKLEARN (JOBLIB)	ONNX RUNTIME
Latência média	0,614 ms	0,083 ms
Latência P95	1,049 ms	0,141 ms
Tamanho do artefato	310 KB	208 KB

7,4x
mais rápido na latência média
inferência só com `onnxruntime`

✓ **Validação honesta:** 98,1% de concordância com o sklearn no test set — as 32 divergências são **empates técnicos** (margem média de 0,019 entre as duas classes mais prováveis), documentadas no benchmark.

RESULTADO · STAR : R



O que Entregamos

0,768

Accuracy (teste)

0,750

Macro F1

0,813**Recall urgente** — vs 0,000
do baseline**0,083 ms**

latência média (ONNX)

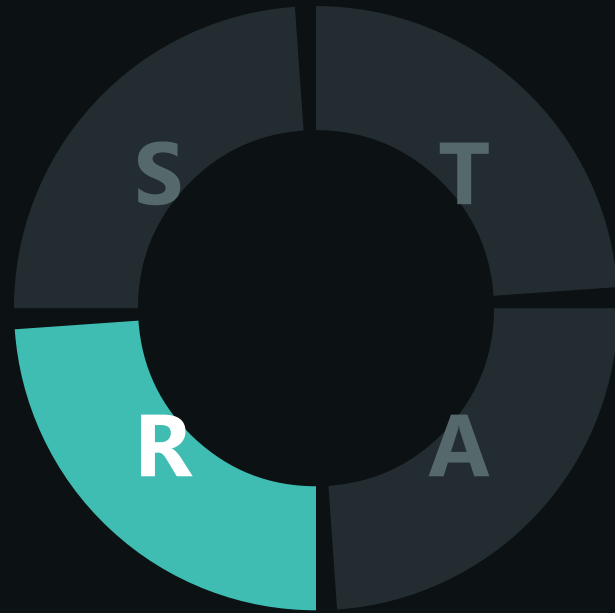
Requisitos do challenge — todos cumpridos

- Modelo NLP leve funcional ☒
- DAG Airflow funcional ☒
- API FastAPI + Dockerfile ☒
- Compose + Grafana (6 painéis) ☒
- CI/CD GitHub Actions ☒
- Otimização ONNX + comparativo ☒

Verificável

- API pública no **Cloud Run** respondendo
- CI verde · stack completa em 2 **docker compose**
- **dvc pull** + **dvc repro** reproduzem tudo

RESULTADO · STAR : R



Conclusões e Próximos Passos

🧠 Aprendizados-chave

- **Pseudo-rotulagem exige auditoria** — o viés que zerava o recall de urgente só apareceu ao processar o corpus completo
- **Quality gate automatizado** vale mais que revisão manual de métricas
- **Separar ambientes** — serving sem torch, treino sem API — mantém a imagem leve e o deploy simples
- **Baselines fortes são o teste de honestidade** do ganho do modelo

✅ Entregues

- Modelo + API + CI + DAG + monitoramento + ONNX
- Documentação completa e reprodução por DVC

📌 Próximo passo

- Validação clínica do threshold com especialistas
- Alertas Grafana sobre o guard-rail de **urgente**